

```

#include <stdio.h>

int main() {
    int m, n, p;

    // Input dimensions
    printf("Enter dimensions m, n, p (for matrices m x n and n x p): ");
    scanf("%d %d %d", &m, &n, &p);

    int A[m][n], B[n][p], result[m][p];

    // Input Matrix A
    printf("Enter elements of matrix A (%dx%d): \n", m, n);
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &A[i][j]);
        }
    }

    // Input Matrix B
    printf("Enter elements of matrix B (%dx%d): \n", n, p);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p; j++) {
            scanf("%d", &B[i][j]);
        }
    }

    // Matrix Multiplication logic
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < p; j++) {
            result[i][j] = 0; // Initialize result matrix with 0
            for (int k = 0; k < n; k++) {
                // Use formula: result[i][j] += A[i][k] * B[k][j]
                result[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    // Display Result
    printf("Resulting Matrix (%dx%d): \n", m, p);
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < p; j++) {
            printf("%d ", result[i][j]); // Moved inside the loop
        }
    }
}

```

```

    }
    printf("\n"); // Newline after each row
}

return 0;
} // Added missing closing bracket
Enter dimensions m, n, p (for matrices m x n and n x p): 3
3
3
Enter elements of matrix A (3x3):
4
5
5
5
5
5
5
456
456
57
Enter elements of matrix B (3x3):
57
78
789
3'
Resulting Matrix (3x3):
308 312 3156
442 468 4734
34200 35568 359784

=== Code Execution Successful ===654
</stdio.h>

```

```
#include <stdio.h>

int main() {
    char str[100];
    int length = 0, i, isPalindrome = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    // 1. Manually calculate length (replaces strlen)
    while (str[length] != '\0') {
        length++;
    }

    // 2. Compare characters from both ends
    for (i = 0; i < length / 2; i++) {
        // Compare character at start (i) with character at end (length - 1 - i)
        if (str[i] != str[length - 1 - i]) {
            isPalindrome = 0; // Not a palindrome
            break;
        }
    }

    if (isPalindrome) {
        printf("%s is a palindrome.\n", str);
    } else {
        printf("%s is not a palindrome.\n", str);
    }

    return 0;
}

</stdio.h>
```

```
Enter a string: 66  
66 is a palindrome.
```

```
=== Code Execution Successful ===
```

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fptr;
    char filename[100], ch;
    int lines = 0, words = 0, characters = 0;
    int in_word = 0;

    printf("Enter filename: ");
    scanf("%s", filename);

    // Open file in read mode
    fptr = fopen(filename, "r");
    if (fptr == NULL) {
        printf("Could not open file %s\n", filename);
        return 1;
    }

    // Read character by character until End Of File (EOF)
    while ((ch = fgetc(fptr)) != EOF) {
        characters++;

        // Count lines
        if (ch == '\n') {
            lines++;
        }

        // Count words: checks for whitespace delimiters
        if (ch == ' ' || ch == '\t' || ch == '\n' || ch == '\0') {
            in_word = 0;
        } else if (in_word == 0) {
            in_word = 1;
            words++;
        }
    }

    // Adjust counts if the file isn't empty but doesn't end with a new line
    if (characters > 0) {
        if (ch != '\n') lines++;
    }

    fclose(fptr);
}

```

```
printf("\nFile Statistics:\n");
printf("Characters: %d\n", characters);
printf("Words:    %d\n", words);
printf("Lines:    %d\n", lines);

return 0;
}
```

```
Enter filename: 57868
Could not open file 57868
```

```
</stdlib.h></stdio.h>=== Code Exited With Errors ===78978
```

```

#include <stdio.h>

// 1. Define the structure
struct Student {
    char name[50];
    int roll_no;
    float marks;
};

int main() {
    // 2. Create an array of structures for 5 students
    struct Student s[5];
    int i, topper_index = 0;

    // 3. Input details for 5 students
    printf("Enter details for 5 students:\n");
    for(i = 0; i < 5; i++) {
        printf("\nStudent %d\n", i + 1);
        printf("Enter Name: ");
        scanf("%s", s[i].name);
        printf("Enter Roll Number: ");
        scanf("%d", &s[i].roll_no);
        printf("Enter Marks: ");
        scanf("%f", &s[i].marks);

        // 4. Logic to find the topper (highest marks)
        if (s[i].marks > s[topper_index].marks) {
            topper_index = i;
        }
    }

    // 5. Display the topper's details
    printf("\n--- TOPPER DETAILS ---\n");
    printf("Name: %s\n", s[topper_index].name);
    printf("Roll No: %d\n", s[topper_index].roll_no);
    printf("Highest Marks: %.2f\n", s[topper_index].marks);

    return 0;
}

```

```
Enter details for 5 students:

Student 1
Enter Name: krishan
Enter Roll Number: 21
Enter Marks: 21

Student 2
Enter Name: yash
Enter Roll Number: 11
Enter Marks: 11

Student 3
Enter Name: abhishek
Enter Roll Number: 44
Enter Marks: 44

Student 4
Enter Name: prince
Enter Roll Number: 66
Enter Marks: 77

Student 5
Enter Name: ok ok
Enter Roll Number: Enter Marks:
--- TOPPER DETAILS ---
Name: prince
Roll No: 66
Highest Marks: 77.00

=== Code Execution Successful ===
```

</stdio.h>


```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[] = {10, 20, 30, 40, 50};
```

```
    int size = 5;
```

```
    int sum = 0;
```

```
    // ptr points to the first element; end points just past the last element
```

```
    int *ptr = arr;
```

```
    int *end = arr + size;
```

```
    // Traverse the array using only pointer comparisons and arithmetic
```

```
    while (ptr < end) {
```

```
        sum += *ptr; // Dereference pointer to get the value
```

```
        ptr++;      // Increment pointer to move to the next address
```

```
    }
```

```
    printf("Sum of array elements: %d\n", sum);
```

```
    return 0;
```

```
Sum of array elements: 150
```

```
=== Code Execution Successful ===
```

```
}</stdio.h>
```